PCA - Practical Implementation

Virtual Labs - Dayalbagh Educational Institute

Step 1: Importing Libraries

 # Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import ipywidgets as widgets
from IPython.display import display
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from ipywidgets import interact, IntSlider
print("Libraries Imported")
```

Output:

```
Libraries Imported
```

Step 2: Loading Dataset

 # Load the Breast cancer Wisconsin dataset

```
df = pd.read_csv('Breast_cancer_Wisconsin_data.csv')
print("Dataset loaded successfully")
```

Output:

```
Dataset loaded successfully
```

Step 3: Data Analysis

 # Display the first 5 rows of the dataset

```
df.head()
```

Output:

	id	radi.	text.	perime	are	smooth.	compact.	concav	concave p	symme.	..	textu.	perime.	are.	smoothn.	compact.	concav.	concave p.	symmet	fractal_dim.	dia
0	84.	17.99	10.38	122.80	100	0.11840	0.27760	0.3001	0.14710	0.2419	...	17.33	184.60	201.	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890	M
1	84.	20.57	17.77	132.90	132	0.08474	0.07864	0.0869	0.07017	0.1812	...	23.41	158.80	195.	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902	M
2	84.	19.69	21.25	130.00	120	0.10960	0.15990	0.1974	0.12790	0.2069	...	25.53	152.50	170.	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758	M
3	84.	11.42	20.38	77.58	386	0.14250	0.28390	0.2414	0.10520	0.2597	...	26.50	98.87	567.	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300	M
4	84.	20.29	14.34	135.10	129	0.10030	0.13280	0.1980	0.10430	0.1809	...	16.67	152.20	157.	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678	M

Output:

```
5 rows x 32 columns
```

Displays the last five rows of the dataset

```
df.tail()
```

Output:

	id	radi.	text.	perime	are	smooth.	compact.	concav	concave p	symme.	..	textu.	perime.	are.	smoothn.	compact.	concav.	concave p.	symmet	fractal_dim.	dia
564	92.	21.56	22.39	142.00	147	0.11100	0.11590	0.2439(	0.13890	0.1726	...	26.40	166.10	202.	0.14100	0.21130	0.4107	0.2216	0.2060	0.07115	M
565	92.	20.13	28.25	131.20	126	0.09780	0.10340	0.1440(	0.09791	0.1752	...	38.25	155.00	173.	0.11660	0.19220	0.3215	0.1628	0.2572	0.06637	M
566	92.	16.60	28.08	108.30	858	0.08455	0.10230	0.0925:	0.05302	0.1590	...	34.12	126.70	112.	0.11390	0.30940	0.3403	0.1418	0.2218	0.07820	M
567	92.	20.60	29.33	140.10	126	0.11780	0.27700	0.3514(	0.15200	0.2397	...	39.42	184.60	182.	0.16500	0.86810	0.9387	0.2650	0.4087	0.12400	M
568	92.	7.76	24.54	47.92	181	0.05263	0.04362	0.0000(	0.00000	0.1587	...	30.37	59.16	268.	0.08996	0.06444	0.0000	0.0000	0.2871	0.07039	B

Output:

```
5 rows x 32 columns
```

Check dataset shape

```
df.shape
```

Output:

```
(569, 32)
```

Displays summary of dataset, including column names, data types, and non-null counts.

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
```

Output:

#	Column	Non-Null Count	Dtype
0	id	569 non-null	int64
1	radius_mean	569 non-null	float64
2	texture_mean	569 non-null	float64
3	perimeter_mean	569 non-null	float64
4	area_mean	569 non-null	float64
5	smoothness_mean	569 non-null	float64
6	compactness_mean	569 non-null	float64
7	concavity_mean	569 non-null	float64
8	concave points_mean	569 non-null	float64
9	symmetry_mean	569 non-null	float64
10	fractal_dimension_mean	569 non-null	float64
11	radius_se	569 non-null	float64
12	texture_se	569 non-null	float64
13	perimeter_se	569 non-null	float64

14	area_se	569 non-null	float64

... 17 more rows (total: 32 rows)

Output:

```
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

Statistical summary

```
df.describe()
```

Output:

	id	radi	text.	perim.	area	smooth.	compac.	conca.	concave .	symme.	..	radi.	textu.	perime.	area	smooth.	compact.	concav.	concave p.	symme.	fractal_di.
co.	5.69.	569.	569..	569.0.	569.	569.00.	569.00.	569.0.	569.00000	569.0.	...	569..	569.0.	569.00.	569.	569.00.	569.000.	569.00.	569.000000	569.0.	569.000000
mea	3.03.	14.1	19.2.	91.96.	654.	0.09636	0.10434	0.088.	0.048919	0.181.	...	16.2.	25.67.	107.26.	880.	0.13236	0.254265	0.27218	0.114606	0.290.	0.083946
std	1.25.	3.52	4.30.	24.29.	351.	0.01406	0.05281	0.079.	0.038803	0.027.	...	4.83.	6.146.	33.602.	569.	0.02283	0.157336	0.20862	0.065732	0.061.	0.018061
mir	8.67.	6.98	9.71.	43.79.	143.	0.05263	0.01938	0.000.	0.000000	0.106.	...	7.93.	12.02.	50.410.	185.	0.07117	0.027290	0.00000	0.000000	0.156.	0.055040
25%	8.69.	11.7	16.1.	75.17.	420.	0.08637	0.06492	0.029.	0.020310	0.161.	...	13.0.	21.08.	84.110.	515.	0.11660	0.147200	0.11456	0.064930	0.250.	0.071460
50%	9.06.	13.3	18.8.	86.24.	551.	0.09587	0.09263	0.061.	0.033500	0.179.	...	14.9.	25.41.	97.660.	686.	0.13130	0.211900	0.22676	0.099930	0.282.	0.080040
75%	8.81.	15.7	21.8.	104.1.	782.	0.10536	0.13040	0.130.	0.074000	0.195.	...	18.7.	29.72.	125.40.	1084	0.14600	0.339100	0.38296	0.161400	0.317.	0.092080
max	9.11.	28.1	39.2.	188.5.	2501	0.16346	0.34540	0.426.	0.201200	0.304.	...	36.0.	49.54.	251.20.	4254	0.22260	1.058000	1.25206	0.291000	0.663.	0.207500

Output:

```
8 rows x 31 columns
```

Counts the frequency of each unique category

```
df['diagnosis'].value_counts()
```

Output:

```
count
diagnosis
B          357
M          212
Name: count, dtype: int64
```

Step 4: Data Preprocessing

Separate features and target variable

```
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']
print("Feature shape:", X.shape)
print("Target shape:", y.shape)
```

Output:

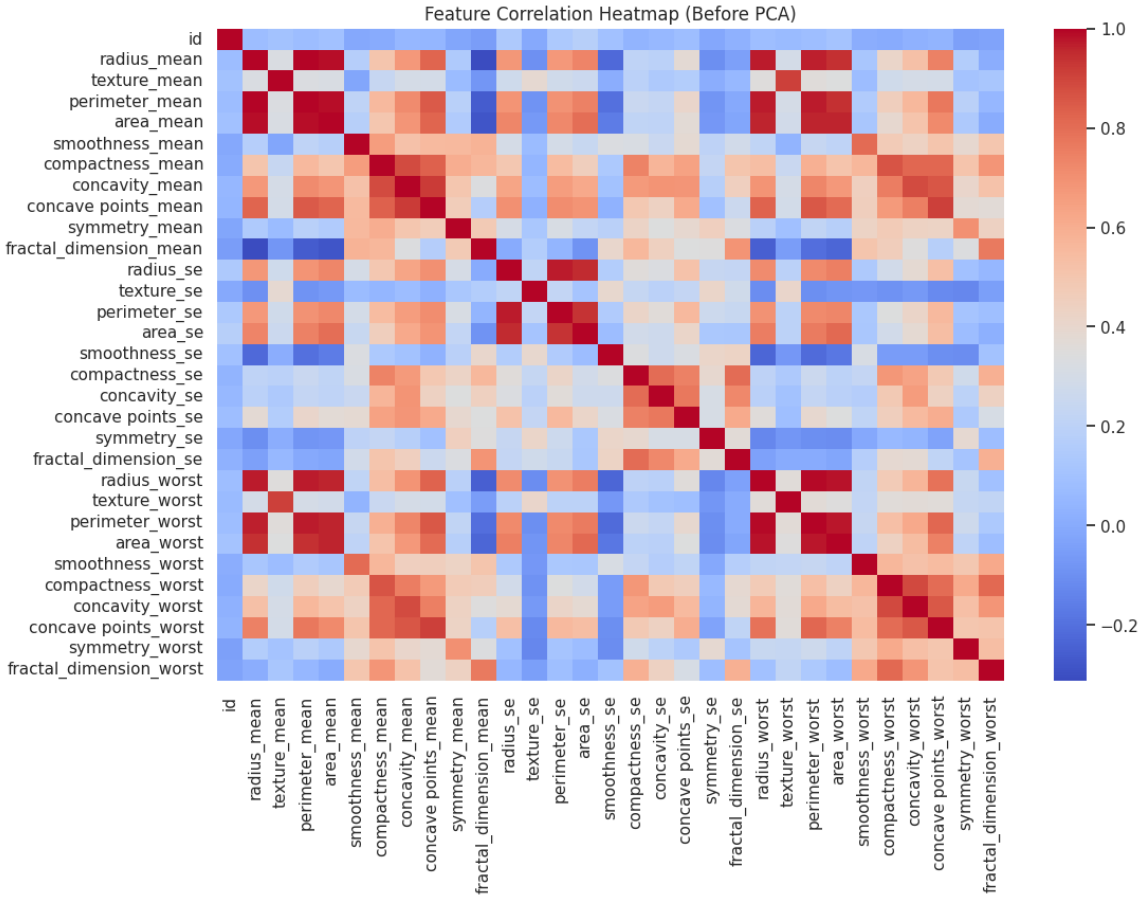
```
Feature shape: (569, 31)
Target shape: (569,)
```

Visualize correlations among features before applying PCA

```
plt.figure(figsize=(12,8))
sns.heatmap(X.corr(), cmap='coolwarm')
plt.title("Feature Correlation Heatmap (Before PCA)")
plt.show()
```

Output:

```
Feature Correlation Heatmap (Before PCA)
```



Standardize the features

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Features standardized using StandardScaler")
```

Output:

```
Features standardized using StandardScaler
```

Step 5: Model Training

Data Splitting

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
print("Data has been Splitted")
```

Output:
```
Data has been Splitted
```

 # Apply PCA, and visualize variance

```
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
print("PCA applied with 95% variance retained.")
```

Output:
```
PCA applied with 95% variance retained.
```

 # Model training using Logistic Regression

```
clf = LogisticRegression(max_iter=2000)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
acc_before = accuracy_score(y_test, y_pred)
clf_pca = LogisticRegression(max_iter=2000)
clf_pca.fit(X_train_pca, y_train)
y_pred_pca = clf_pca.predict(X_test_pca)
acc_after = accuracy_score(y_test, y_pred_pca)
print("Model Training completed")
```

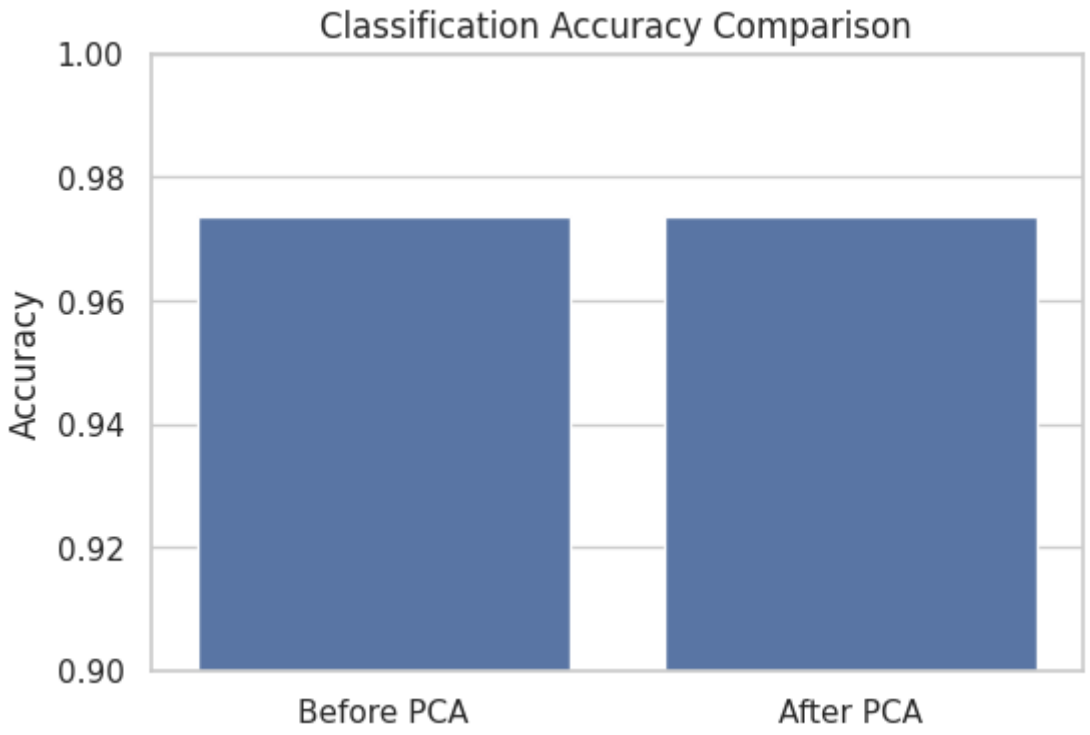
Output:
```
Model Training completed
```

Step 6: Model Evaluation

 # Compare classification accuracy before and after applying PCA

```
print("Accuracy BEFORE PCA:", round(acc_before*100, 2), "%")
pca = PCA(n_components=0.95)
X_pca = pca.fit_transform(X_scaled)
print("Reduced Dimensions:", X_pca.shape[1])
X_train_pca, X_test_pca, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.2, random_state=42, stratify=y
)
print("Accuracy AFTER PCA:", round(acc_after*100, 2), "%")
plt.figure(figsize=(6,4))
sns.barplot(
    x=['Before PCA', 'After PCA'],
    y=[acc_before, acc_after]
)
plt.ylabel("Accuracy")
plt.title("Classification Accuracy Comparison")
plt.ylim(0.9, 1.0)
plt.show()
```

Output:
```
Accuracy BEFORE PCA: 97.37 %
Reduced Dimensions: 11
Accuracy AFTER PCA: 97.37 %
```



 # Apply PCA and compute feature loadings for each principal component

```
pca = PCA(n_components=10)
X_pca = pca.fit_transform(X_scaled)
loadings = pd.DataFrame( pca.components_.T, columns=[f"PC{i+1}" for i in range(10)],
    index=X.columns)
print("PCA feature loadings calculated for the top 10 principal components.")
```

Output:

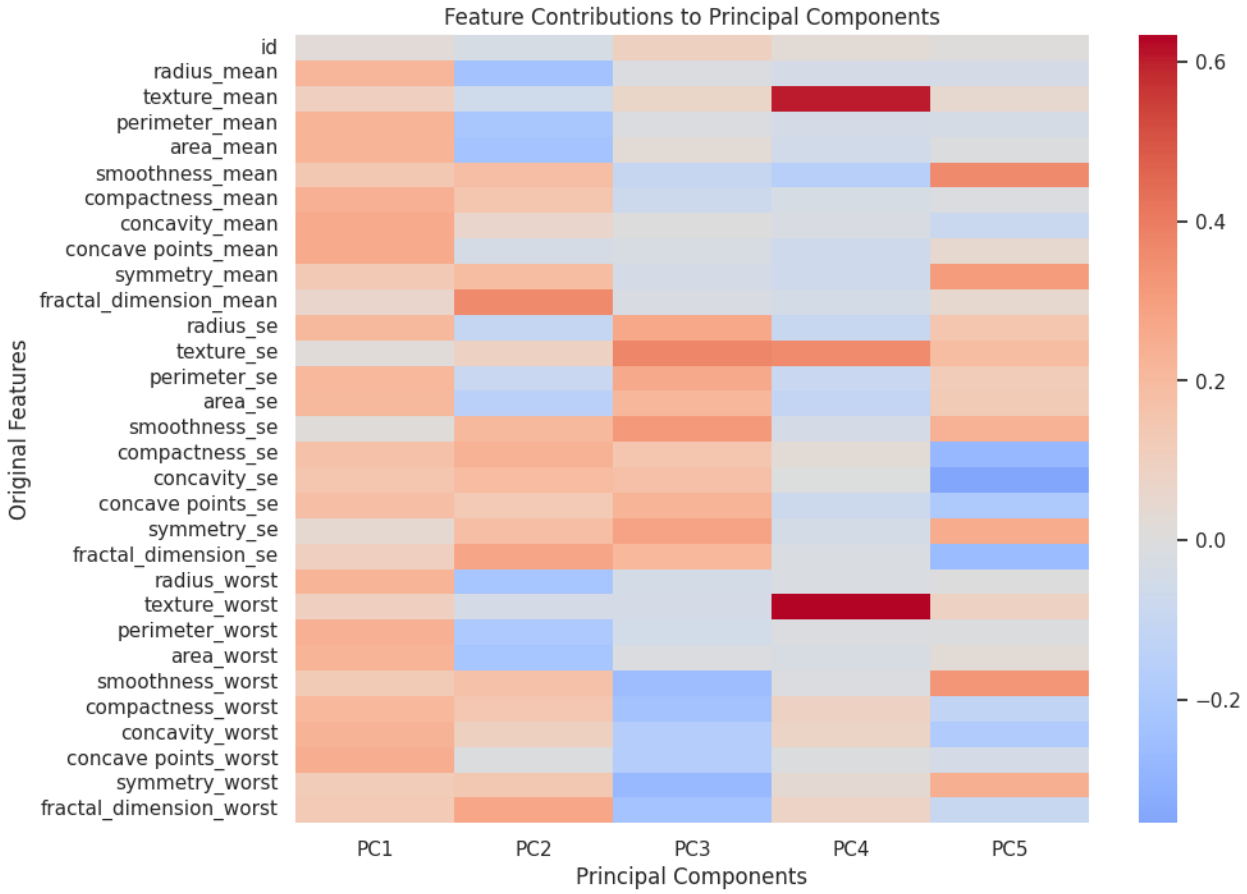
```
PCA feature loadings calculated for the top 10 principal components.
```

 # Visualize feature contributions to the first few principal components using a heatmap

```
plt.figure(figsize=(10,8))
sns.heatmap(loadings.iloc[:, :5], cmap="coolwarm",center=0)
plt.title("Feature Contributions to Principal Components")
plt.xlabel("Principal Components")
plt.ylabel("Original Features")
plt.show()
```

Output:

```
Feature Contributions to Principal Components
```



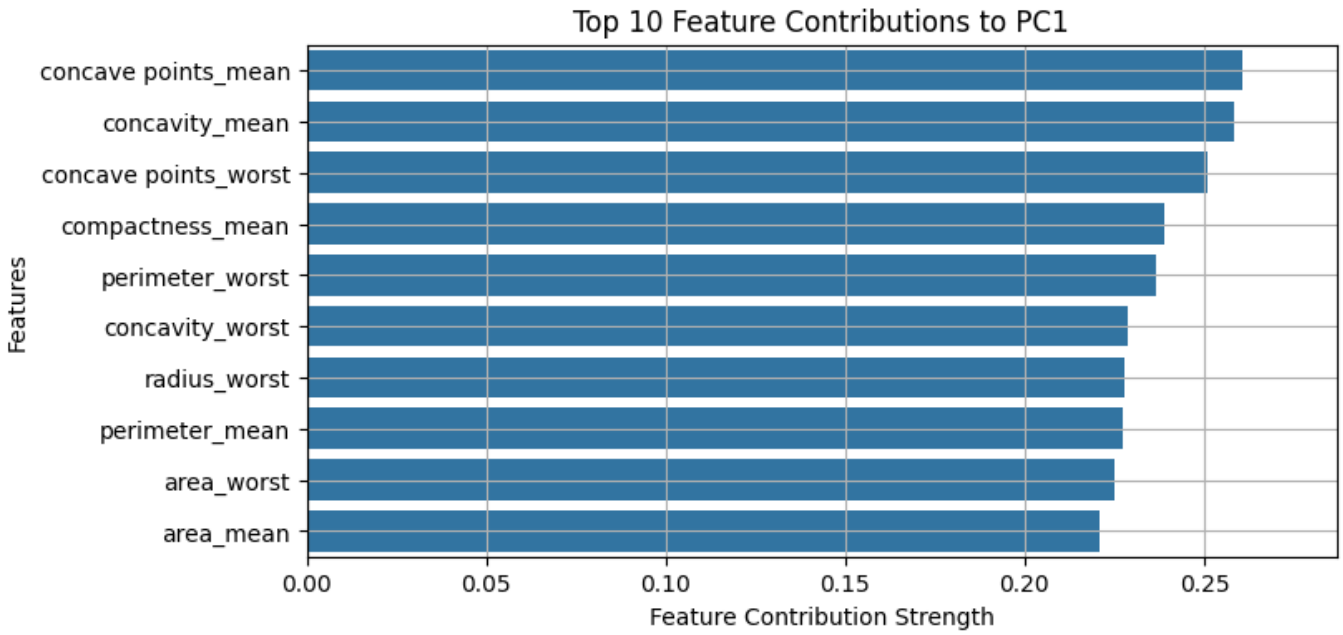
 # Interactively analyze and visualize top feature contributions for a selected principal component

```
def interactive_pca(pc_num=1):
    pc = f"PC{pc_num}"
    print(f"Principal Component: {pc}")
    print(f"Explained Variance Ratio: {pca.explained_variance_ratio_[pc_num-1]:.4f}")
    print(f"Cumulative Variance till {pc}:
    {pca.explained_variance_ratio_[:pc_num].sum():.4f}")
    top_features = loadings[pc].abs().sort_values(ascending=False).head(10)
    plt.figure(figsize=(8,4))
    sns.barplot( x=top_features.values, y=top_features.index)
    plt.xlim(0, top_features.max() * 1.1)
    plt.xlabel("Feature Contribution Strength")
    plt.ylabel("Features")
    plt.title(f"Top 10 Feature Contributions to {pc}")
    plt.grid(True)
    plt.show()

widgets.interact(interactive_pca,
    pc_num=widgets.IntSlider(min=1,max=10,step=1,value=1,description="PCA Component"));
```

Output:

```
Select any PCA component to visualize its feature contributions:
PCA Comp...
1
Principal Component: PC1
Explained Variance Ratio: 0.4286
Cumulative Variance till PC1: 0.4286
```



Interactive Features (Static View)

Interactive Feature: PCA Component Variance Viewer

In the simulation, a slider allows you to select PCA components 1-10 and view feature contributions. Below is the variance data for all components:

Component	Explained Variance Ratio	Cumulative Variance
PC1	0.4286	0.4286
PC2	0.1838	0.6124
PC3	0.0915	0.7039
PC4	0.0639	0.7678
PC5	0.0532	0.8210
PC6	0.0398	0.8608
PC7	0.0316	0.8924
PC8	0.0217	0.9140
PC9	0.0149	0.9289
PC10	0.0130	0.9419